

2025
International Internship Pilot Program
in Taiwan

Final Report

Name : Beron Zapata Juan Esteban

Internship Inst. : National Central University

Name of PI/Department/School : Min-Chun Pan Ph.D.

Mechanical Engineering Department

Submission Date : 26/02/2026

Development of a ROS 2-Based Inertial Navigation and Positioning Framework on NVIDIA Jetson for Autonomous Systems

January 26, 2026

Abstract

This document presents the final technical report for a three-month research internship at the SenDiMoniProg Laboratory under the International Internship Pilot Program (IIPP). The project focuses on the design, implementation, and evaluation of a distributed perception architecture combining an inertial measurement unit (IMU) and an Intel RealSense D405 depth camera. All sensing and pre-processing are executed on an NVIDIA Jetson platform running ROS 2 inside Docker containers, while visualization and higher-level processing are performed remotely on a second machine connected through Tailscale's virtual private network.

The report describes the full evolution of the system: from an initial USB-based IMU prototype, through a Bluetooth serial link, then to a WebSocket bridge over WiFi, and finally to a depth-camera-enabled architecture exposing 2D images and 3D point clouds through a lightweight web server. We analyse latency, bandwidth consumption, and numerical stability of the orientation estimates obtained via the Madgwick filter, and we discuss the impact of transport choices on usability in realistic wireless scenarios.

The work demonstrates that a ROS 2-Docker stack, carefully modularized into nodes and packages, enables robust experimentation with heterogeneous sensors while keeping network traffic under control. The outcomes include a reusable code base for wireless IMU and depth camera streaming, along with a set of design guidelines for future deployments on field robots and aerial platforms.

Contents

1	Introduction	4
2	Internship Reflection and Professional Development	4
2.1	Motivation: intention to come to Taiwan for this internship	4
2.2	Gains and experience acquired from the internship	5
2.2.1	Technical gains	5
2.2.2	Research and academic gains	5
2.3	Life in Taiwan and strategies to deal with difficulties	5
2.4	Most impressive aspects and reasons to recommend the internship	6
2.5	Would I participate again?	7
2.6	PI and institutes interacted with during the internship	7
2.7	Summary of outcomes and professional impact	7
2.8	Visits and networking	8
2.9	Presentation of the project	8
2.10	Personal and academic impact	8
3	System Architecture	9
3.1	Hardware components	9
3.2	Software stack	12
4	Related Work and Theoretical Background	12
4.1	Orientation representation and IMU models	12
4.2	Madgwick filter overview	13
4.3	Depth sensing and point–cloud generation	13
5	Repository Structure and ROS 2 Packages	14
6	Methods	16
6.1	IMU acquisition over USB and Bluetooth	16
6.2	Madgwick orientation fusion	18
6.3	WebSocket bridge and Docker networking	18
6.4	RealSense D405 integration	19
6.5	Point–cloud generation and web server	20
7	Evaluation Methodology	21
8	Results	22
8.1	Network performance	22
8.2	IMU fusion quality	24
8.3	Depth and point–cloud quality	25

8.4 Remote visualization in RViz 2	26
9 Discussion	26
9.1 Advantages of the ROS 2 node architecture	26
9.2 Challenges encountered	26
9.3 Comparison of communication modes	27
10 Reflection on the Last Three Months of IIPP	27
11 Conclusion and Future Work	28
A Key ROS 2 Topics and Message Types	28
B Simplified Pseudo-code for the IMU Publisher	29
C Docker Image Structure	30

1 Introduction

Modern robotic systems require reliable state estimation and dense perception to operate safely in real environments. In aerial robotics and autonomous navigation, this typically involves combining inertial measurements (IMU) with vision and depth sensing, while maintaining low latency and robustness under limited computational and network resources.

This report summarizes the work conducted during the last three months of the International Internship Pilot Program (IIPP) at the SenDiMoniProg Laboratory (National Central University, Taiwan), funded by the National Science and Technology Council (NSTC). The main contribution of this internship was the design and validation of a modular ROS2 sensing architecture that evolved from a wired IMU prototype to a fully remote, containerized multi-sensor pipeline.

The final system integrates an ESP32-based IMU streaming over Bluetooth to an embedded computer, a WebSocket bridge for remote visualization, and an Intel RealSense D405 camera producing depth images and colored point clouds. Performance was evaluated in terms of stability, latency, and bandwidth usage under realistic network conditions, providing practical insights into the trade-offs of wireless sensing in robotics.

2 Internship Reflection and Professional Development

2.1 Motivation: intention to come to Taiwan for this internship

My primary intention to come to Taiwan under the International Internship Pilot Program (IIPP) was to engage in a high-impact research environment focused on robotics, sensing, and autonomous systems. As my long-term professional objective is to specialize in advanced robotics and autonomous aerial platforms, I sought an internship that would allow me to (i) work with real sensors and embedded hardware, (ii) develop reproducible robotic software pipelines, and (iii) strengthen my academic foundations for future graduate studies.

Taiwan represents an exceptional ecosystem for technology and research due to its global leadership in semiconductors, embedded systems, and instrumentation. In particular, the opportunity to work at National Central University (NCU) within the SenDiMoniProg Laboratory, and to interact with the broader research infrastructure supported by NSTC, made this internship uniquely aligned with both my academic and professional trajectory.

From the beginning, I defined this internship as a systems engineering challenge: to evolve from a basic sensor acquisition prototype into a complete modular sensing pipeline, capable of remote inspection and deployment across heterogeneous computing devices. The internship therefore served not only as a research experience, but also as an intensive training period in applied robotics engineering.

2.2 Gains and experience acquired from the internship

The internship produced measurable gains across technical competencies, engineering methodology, and academic communication. The most significant outcome was the development of a complete sensing and visualization pipeline that transitioned through multiple communication paradigms, from local wired acquisition to fully remote networked operation.

2.2.1 Technical gains

During the last three months, I acquired practical experience in the following areas:

- **ROS 2 development and debugging:** creation and validation of publisher/subscriber nodes, topic inspection, message formatting, QoS awareness, and multi-machine operation.
- **Wireless sensing architectures:** implementation and evaluation of Bluetooth serial acquisition (ESP32 → Jetson), WiFi connectivity, and WebSocket streaming for remote visualization.
- **Sensor integration:** reading and validating IMU measurements (accelerometer, gyroscope, magnetometer, pressure, temperature) and integrating Intel RealSense D405 for depth sensing and point cloud generation.
- **Containerization and reproducibility:** Docker image creation for embedded and workstation environments, dependency management, and stable runtime deployment.
- **Performance evaluation:** identification of bottlenecks such as bandwidth saturation, frame drops, latency, and visualization instability under real network constraints.

2.2.2 Research and academic gains

Beyond implementation, I gained experience in documenting engineering decisions, reporting results rigorously, and presenting work to a technical audience. This included designing evaluation scenarios, interpreting failures, and justifying architectural changes based on empirical evidence rather than assumptions.

2.3 Life in Taiwan and strategies to deal with difficulties

Living in Taiwan during the internship was an important part of the experience, providing both professional growth and personal development. Adapting to a new environment required managing daily logistics independently while maintaining consistent research productivity.

Several difficulties emerged naturally in an international research setting:

- **Cultural and language barriers:** addressed through proactive communication, careful planning, and willingness to learn from local context.
- **Hardware–software integration challenges:** resolved by adopting systematic debugging practices, validating each subsystem independently, and using controlled test cases.
- **Remote development constraints:** mitigated through Dockerization, network overlay tools, and remote visualization pipelines that reduced the need for local physical access.

A key lesson was the importance of disciplined engineering workflows: isolating variables, logging outputs, validating assumptions, and maintaining clean repository structure. This approach was essential to overcome failures such as unstable network streaming, inconsistent sensor formats, and visualization glitches.

2.4 Most impressive aspects and reasons to recommend the internship

The aspect that impressed me most was the combination of strong institutional support and meaningful technical freedom. The IIPP program enabled research interns to work on real engineering problems with direct relevance to modern robotics, while also providing opportunities for professional networking and academic presentation.

One of the most impactful moments was the IIPP Scholars and Research Interns Meeting (December 15), which included visits to institutions of strategic importance:

- **TSMC Museum visit:** highlighted how long-term vision and research investment can transform a country into a global technological leader, reinforcing the value of sustained innovation.
- **NCIR visit:** demonstrated Taiwan’s strength in optical systems, precision instrumentation, and technology development pipelines relevant to sensing and robotics.

In addition, the opportunity to present the project alongside my supervisor, Prof. Min-Chun Pan, strengthened my scientific communication skills. The feedback and interest received from other interns provided both validation and new ideas for future extensions, such as GNSS integration and multi-sensor synchronization.

For these reasons, I strongly recommend this internship to students seeking serious exposure to robotics research, embedded systems, and international collaboration.

2.5 Would I participate again?

Yes, I would participate again without hesitation. The program provided a rare combination of academic rigor, hands-on engineering, and international exposure. The experience accelerated my learning curve in robotics significantly and helped solidify my professional direction toward autonomous systems research.

Unlike short or purely observational internships, this program enabled full ownership of a technically complex pipeline, requiring real implementation, debugging, and evaluation. This depth of responsibility is precisely what makes the IIPP internship highly valuable and worth recommending.

2.6 PI and institutes interacted with during the internship

During the internship, I worked primarily under the supervision of Prof. Min-Chun Pan at the SenDiMoniProg Laboratory, National Central University (NCU), Taiwan. The research environment provided access to both hardware resources and expert guidance necessary for multi-sensor integration.

Additionally, during the December 15 IIPP event, I interacted with researchers and staff at:

- **Taiwan Semiconductor Manufacturing Company (TSMC)** – museum visit and technology overview.
- **National Center for Instrumentation Research (NCIR)** – exposure to optical systems, instrumentation, and sensing research.

These interactions contributed to a broader understanding of Taiwan's research ecosystem and reinforced the importance of robust sensing and calibration in high-precision systems.

2.7 Summary of outcomes and professional impact

This internship has become a cornerstone of my professional development. It strengthened my technical abilities in robotics software engineering and provided a research-driven context for applying those skills to real sensing problems. The progression from local wired sensing to fully remote wireless multi-sensor visualization reflects a realistic robotics development trajectory and prepares me for future graduate studies and specialization in autonomous systems.

To summarize the experience at a high level, Table 1 presents the main outcomes of the internship.

Table 1: Summary of internship outcomes (IIPP, last three months).

Category	Outcome
Technical implementation	ROS 2 IMU pipeline, RealSense D405 integration, WebSocket streaming, Dockerized deployment
Wireless evolution	USB → Bluetooth → WiFi → WebSockets
Evaluation	Latency, bandwidth constraints, visualization stability, failure modes under real networks
Research skills	Experimental design, documentation, academic writing, presentation to technical audiences
Professional growth	Increased autonomy, debugging discipline, cross-cultural collaboration, long-term research motivation

2.8 Visits and networking

On December 15, IIPP scholars and research interns participated in a meeting that included visits to Taiwan Semiconductor Manufacturing Company (TSMC) and the National Center for Instrumentation Research (NCIR). The tour of TSMC’s museum illustrated how the long-term vision of Dr. Morris Chang transformed Taiwan into a global semiconductor hub. At NCIR, the strong focus on optics, precision instrumentation and satellite payloads highlighted the importance of accurate sensing and calibration, reinforcing the motivation for precise depth and IMU measurements in our own project.

2.9 Presentation of the project

During the same event the SenDiMoniProg laboratory was invited as a special guest. Together with my supervisor, Prof. Min-Chun Pan, I presented the objectives and current status of the wireless IMU + camera architecture. This gave an opportunity to refine the problem statement, obtain critical feedback and identify possible extensions, such as the integration of GNSS and multi-camera rigs.

2.10 Personal and academic impact

From an academic perspective, the internship allowed me (Juan Esteban Beron) to transition from a primarily aeronautical background into practical robotics and embedded sensing. Working daily with ROS 2, Docker, point clouds and network debugging tools has built a strong foundation for future graduate studies in autonomous systems. Personally,

the experience of collaborating in an international team and communicating results in English has been invaluable.

3 System Architecture

3.1 Hardware components

Table 2 summarizes the main hardware components used in the project. The sensing and low-level processing occur on an NVIDIA Jetson board running Ubuntu and ROS 2 inside Docker. An ESP32 board, instrumented with IMU and pressure sensors, communicates via Bluetooth. A RealSense D405 camera provides depth and color images. The remote workstation (Math and Visualization Processing Unit, MVPU) runs RViz 2 and additional analysis tools.

Table 2: Hardware components of the wireless sensing architecture.

Component	Role	Description
NVIDIA Jetson Nano	Embedded host	Runs Dockerized ROS 2 nodes, connects to sensors and Tailscale network.
ESP32 board + sensors	IMU/pressure source	Streams accelerometer, gyroscope, magnetometer, temperature and altitude data via Bluetooth Serial.
RealSense D405	Depth camera	Provides synchronized color and depth images at VGA resolution for indoor scenes.
Raspberry Pi 4 + GPS	GNSS module (future work)	Intended to provide absolute positioning data; used in preliminary tests.
Remote workstation (MVPU)	Visualization and analysis	Connects over Tailscale VPN, runs RViz 2 and Python tools, receives ROS 2 topics via wireless link.
WiFi routers + Tailscale	Network backbone	Provide local wireless network and secure virtual private network between Jetson and MVPU.

Figure 1 reproduces the high-level roadmap of the project, illustrating the successive stages from system foundation to autonomy and evaluation.

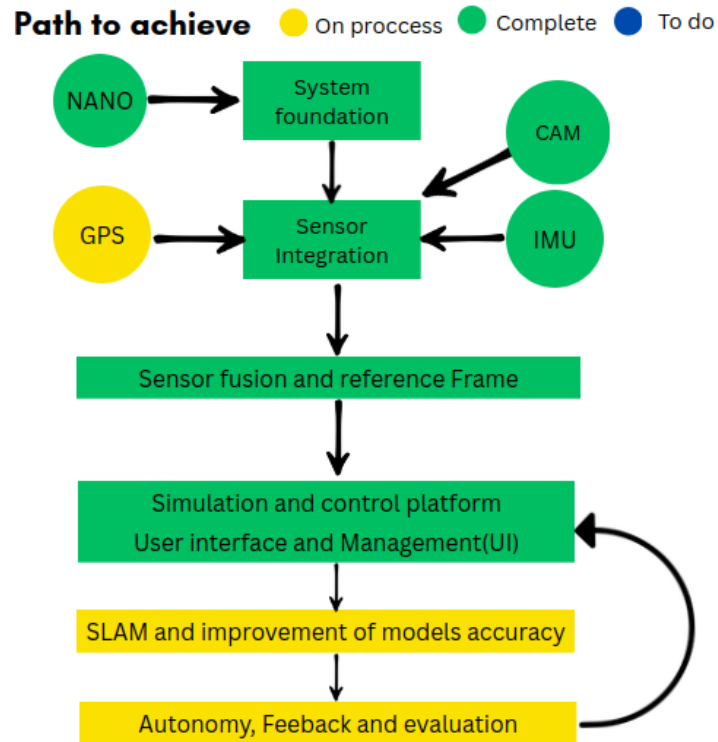


Figure 1: Path to achieve full autonomy, from sensor integration to SLAM and feedback control.

Figure 2 illustrates the final distributed architecture with Docker and Tailscale.

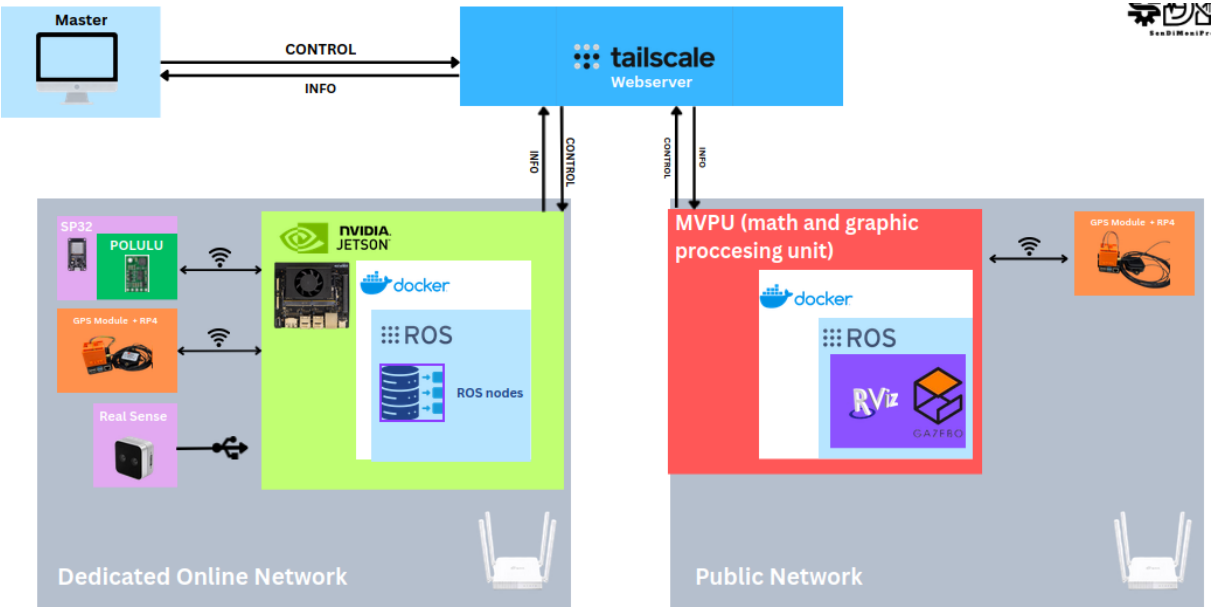


Figure 2: Conceptual architecture: sensors connected to the Jetson (left) stream data through ROS 2 and WebSockets over Tailscale to the remote workstation (right).

3.2 Software stack

The software stack combines ROS 2 (Kilted distribution), Python and C++ nodes, Docker containers and networking tools. Table 3 summarizes the main software technologies.

Table 3: Software stack used in the project.

Technology	Role
ROS 2 (Kilted)	Middleware for distributed nodes, topics and parameters.
Docker	Containerization of the ROS 2 environment on Jetson and MVPU.
Python 3.12	Implementation language for IMU and WebSocket nodes.
C++ (RealSense ROS 2 driver)	Native node providing depth and color streams.
Intel librealsense 2	Low-level driver for the D405 camera.
Tailscale	Mesh VPN providing secure IP connectivity between devices.
Flask + OpenCV	Lightweight web server for image and point-cloud streaming.
RViz 2	3D visualization of IMU frames, depth maps and point clouds.

4 Related Work and Theoretical Background

4.1 Orientation representation and IMU models

Orientation in three dimensions can be represented using rotation matrices, Euler angles, axis-angle pairs or quaternions. For filtering and numerical integration, quaternions are often preferred due to the absence of gimbal lock and the efficiency of multiplication. An IMU typically outputs specific force (accelerometer), angular rate (gyroscope) and, sometimes, magnetic field (magnetometer). The measurements are corrupted by bias, scale factors and noise. Table 4 summarizes the simplified measurement model adopted in this project.

Table 4: Simplified IMU measurement model.

Signal	Model
Specific force	$\mathbf{a}_m = \mathbf{R}^\top (\mathbf{a} - \mathbf{g}) + \mathbf{b}_a + \mathbf{n}_a$
Angular rate	$\boldsymbol{\omega}_m = \boldsymbol{\omega} + \mathbf{b}_g + \mathbf{n}_g$
Magnetic field	$\mathbf{m}_m = \mathbf{R}^\top \mathbf{m}_{\text{world}} + \mathbf{b}_m + \mathbf{n}_m$

Here $\mathbf{R}$ is the rotation matrix from world to body frame, $\mathbf{g}$ is gravity, $\mathbf{b}$. denote biases and $\mathbf{n}$. represent zero-mean noise. The Madgwick filter used in this project exploits the direction of gravity and the Earth’s magnetic field to correct drift from gyroscope integration.

4.2 Madgwick filter overview

The Madgwick filter formulates orientation estimation as a gradient-descent problem in quaternion space. Given a current orientation estimate $\hat{q}$ and gyroscope measurement $\boldsymbol{\omega}_m$, the quaternion is first propagated using a kinematic equation. Then, an error term based on accelerometer and magnetometer measurements is computed and used to correct the estimate.

Although a full derivation is beyond the scope of this report, Table 5 outlines the main operations performed at each time step.

Table 5: Main computational steps of the Madgwick filter.

Step	Description
1. Normalization	Normalize accelerometer and magnetometer vectors to unit length.
2. Objective function	Compute error between expected and measured gravity and magnetic field directions.
3. Gradient descent	Compute gradient of the error with respect to the quaternion components.
4. Correction term	Scale gradient by factor β to obtain a correction quaternion derivative.
5. Gyro integration	Integrate gyroscope readings into quaternion derivative.
6. Update and normalize	Update quaternion and renormalize to maintain unit norm.

4.3 Depth sensing and point-cloud generation

Structured-light and stereo depth cameras infer range by analysing disparities between projected or observed patterns. The D405 is a stereo depth camera optimized for short

ranges; its ROS 2 driver outputs depth images where each pixel stores distance along the camera optical axis.

Point-cloud generation involves projecting each pixel (u, v) with depth z into 3D coordinates using the pinhole model. Table 6 summarizes the equations.

Table 6: Projection from depth image to 3D points.

Quantity	Equation
Normalized coordinates	$x_n = (u - c_x)/f_x, \quad y_n = (v - c_y)/f_y$
3D point	$X = zx_n, \quad Y = zy_n, \quad Z = z$
Color mapping	(R, G, B) taken from registered color image at (u, v)

5 Repository Structure and ROS 2 Packages

The Git repository `SenDiMoniProg-IMU` groups all the components required for the wireless IMU+camera pipeline. Figure 3 shows a schematic representation of the directory tree.

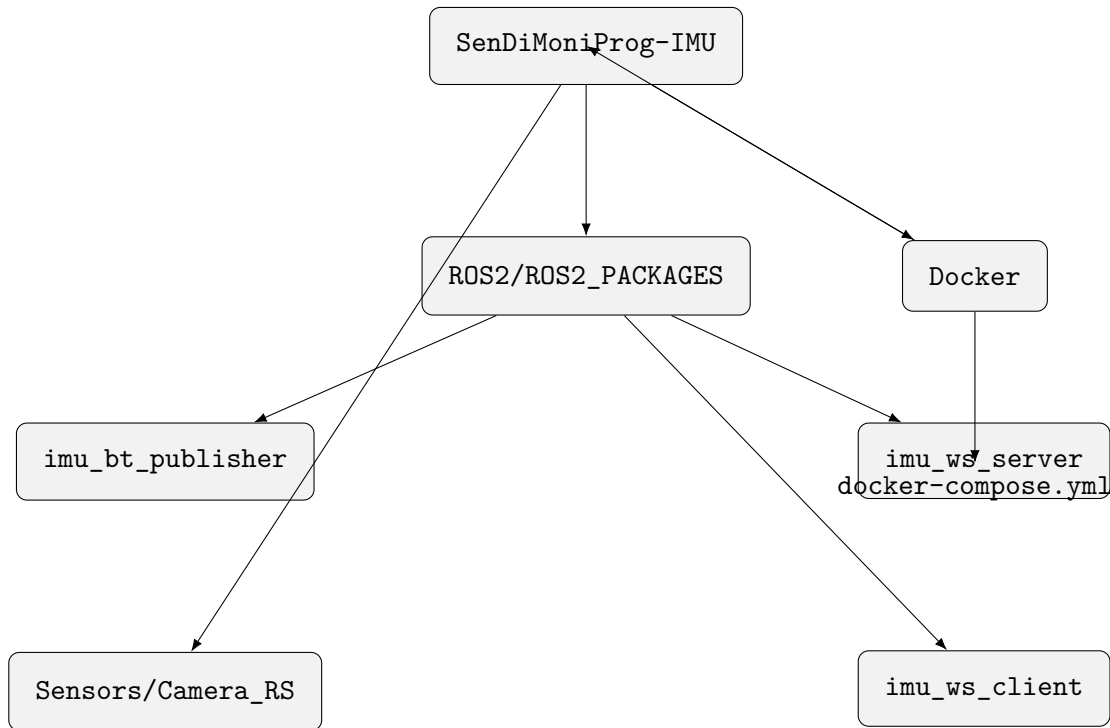


Figure 3: Simplified view of the repository structure.

Table 7 lists the main ROS 2 packages and nodes, with short descriptions.

Table 7: ROS 2 packages and principal nodes in the repository.

Package	Language	Node(s) and purpose
<code>imu_bt_publisher</code>	Python	<code>bluetooth_imu_publisher</code> : reads CSV frames from <code>/dev/rfcomm0</code> , publishes <code>sensor_msgs/Imu</code> , magnetic field, pressure, temperature and altitude topics, and computes Madgwick orientation.
<code>imu_ws_server</code>	Python	<code>imu_ws_server_node</code> : subscribes to <code>/imu/data</code> and exposes the data via a WebSocket server for remote ROS 2 clients or custom dashboards.
<code>imu_ws_client</code>	Python	<code>imu_ws_client_node</code> : connects to the WebSocket server from the MVPU, republishes the IMU data locally on <code>/imu/data</code> , enabling RViz 2 visualization without running ROS 2 on the Jetson.
<code>realsense2_camera</code>	C++	<code>camera/d405</code> : vendor-provided driver node that publishes color and depth images, camera info, and extrinsic parameters.
<code>depth_image_proc</code>	C++	Point cloud nodes that convert registered color and depth images into <code>sensor_msgs/PointCloud2</code> for RViz 2 and web streaming.
<code>realsense_ros_web_bridge</code>	Python	Bridge node that subscribes to image and point-cloud topics and forwards data to a Flask-based web server.

To make the data flow more explicit, Table 8 enumerates the key ROS 2 topics used in the final system.

Table 8: Main ROS 2 topics in the final configuration.

Topic	Type	Producer / consumer
/imu/data	sensor_msgs/Imu	Produced by <code>bluetooth_imu_publisher</code> , consumed by WebSocket server and RViz 2.
/camera/d405/color/image_rect_raw	sensor_msgs/Image	Produced by <code>realsense2_camera</code> , used by web bridge and <code>depth_image_proc</code> .
/camera/d405/depth/image_rect_raw	sensor_msgs/Image	Depth image registered to color frame.
/d405/depth/color/points	sensor_msgs/PointCloud2	Colored point cloud produced by <code>point_cloud_xyzrgb_node</code> .
/tf	tf2_msgs/TFMessage	Coordinate frame transforms for IMU, camera and world frames.

6 Methods

6.1 IMU acquisition over USB and Bluetooth

The initial prototype used a wired USB connection between the ESP32 and the Jetson. The microcontroller streamed CSV lines containing twelve floating-point values: three accelerations in g , three angular rates in degrees per second, three magnetic field components in microtesla, pressure in hPa, temperature in degrees Celsius, and altitude in meters.

Table 9 summarizes the content of each CSV line.

Table 9: Content of a single IMU CSV frame.

Field	Units	Description
ax_g, ay_g, az_g	<i>g</i>	Specific force along X, Y, Z axes.
gx_dps, gy_dps, gz_dps	deg/s	Angular velocity around X, Y, Z.
mx_uT, my_uT, mz_uT	μ T	Magnetic field components.
p_hpa	hPa	Atmospheric pressure.
t_C	°C	Temperature.
alt_m	m	Altitude estimate based on barometric pressure.

The final IMU publisher node was rewritten to use Bluetooth Serial via the device `/dev/rfcomm0`. The core structure remained identical: a line buffer, a parser that validates the field count, and a function that maps the parsed values into ROS 2 messages.

Table 10: Comparison of transport options for IMU streaming.

Transport	Medium	Typical latency	Remarks
USB (CDC)	Wired	< 1 ms	Physically reliable but limited mobility.
Bluetooth SPP	Wireless	5–15 ms	Sufficient for 50 Hz IMU, simplifies mounting on moving platforms.
WiFi + WebSocket	Wireless	10–30 ms	Adds overhead but enables multi-client access and routing via Tailscale.

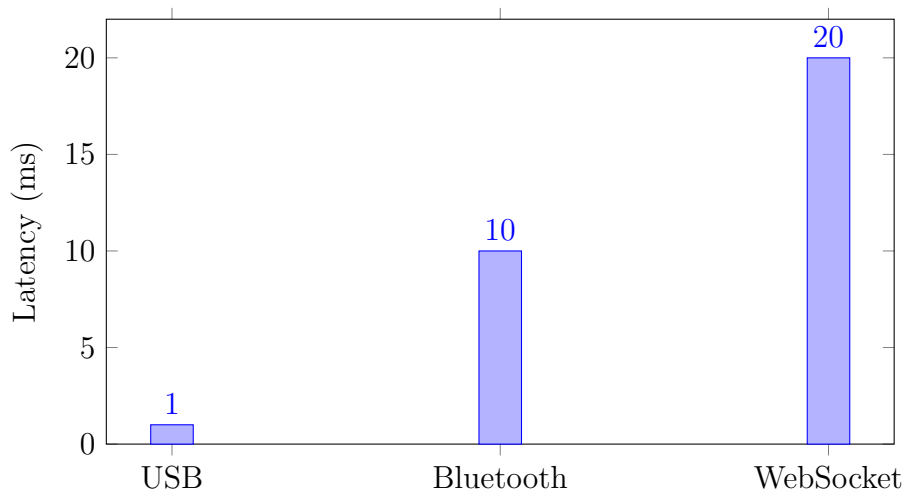


Figure 4: Approximate end-to-end latency for different IMU transport layers.

6.2 Madgwick orientation fusion

To obtain orientation in quaternion form, the project relied on the Madgwick filter implemented in the `ahrs` Python library. Table 11 summarizes the main parameters.

Table 11: Madgwick filter configuration.

Parameter	Value	Description
Sampling frequency	50 Hz	Update rate, estimated from inter-sample intervals.
Damping factor β	0.2	Trade-off between responsiveness and noise rejection.
Input signals	Accelerometer, gyroscope, magnetometer	All three modalities are used via the <code>updateMARG</code> call.
Initialization	Identity quaternion	Assumes initial alignment with world frame.

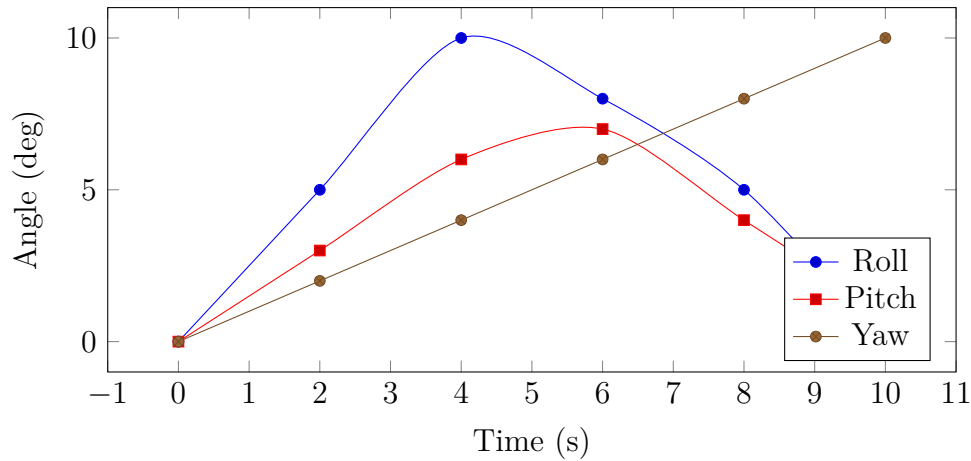


Figure 5: Illustrative evolution of fused Euler angles during an experiment.

6.3 WebSocket bridge and Docker networking

The `imu_ws_server` node runs inside the Jetson Docker container. It subscribes to `/imu/data` and launches an asynchronous WebSocket server using the `websockets` Python library. Clients connect by specifying the Jetson’s Tailscale IP address and port 8765; on

the MVPU side, `imu_ws_client` reconnects automatically when the link is temporarily lost.

Figure 6 presents a simplified state machine of the client node.

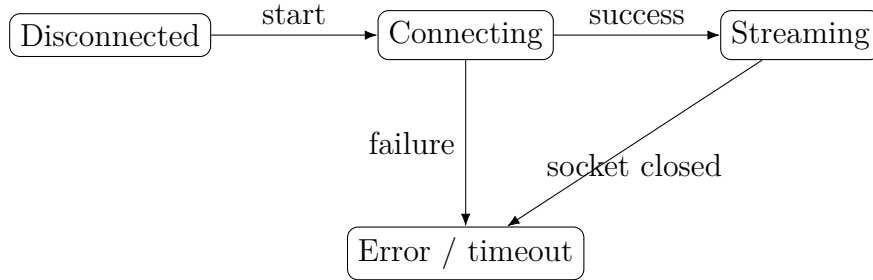


Figure 6: State machine for the WebSocket IMU client.

6.4 RealSense D405 integration

The Intel RealSense D405 is a short-range depth camera optimized for distances below one meter. The ROS 2 integration is based on the `realsense2_camera` driver, which publishes color and depth images and the corresponding camera information.

Table 12 lists the main configuration parameters used during the experiments.

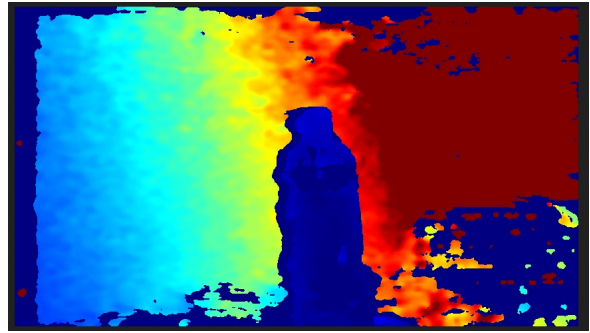
Table 12: RealSense D405 configuration for indoor experiments.

Parameter	Value	Notes
Color resolution	848 × 480 @ 30 fps	RGB8 format for web streaming and RViz 2 texture.
Depth resolution	848 × 480 @ 30 fps	16-bit depth (Z16) aligned to color frame.
Minimum range	0.07 m	According to manufacturer, verified experimentally.
Maximum reliable range	1.0 m	Beyond this, noise increases sharply in our scenes.
Point cloud source	Registered color + depth	Generated via <code>depth_image_proc</code> and published on <code>/d405/depth/color/points</code> .

Figure 7 shows representative color and depth images captured during calibration and testing.



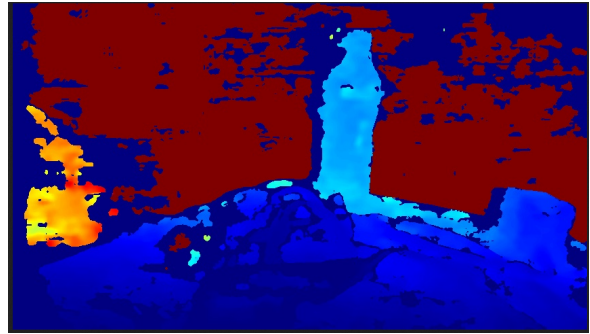
(a) Color image of the classroom scene.



(b) Depth map with colormap.



(c) Color image with IMU prototype in foreground.



(d) Corresponding depth map highlighting the bottle.

Figure 7: Example RealSense D405 color and depth images used in the experiments.

6.5 Point-cloud generation and web server

The creation of colored point clouds relies on the `depth_image_proc` package. The node `point_cloud_xyzrgb_node` subscribes to the registered depth image, color image and camera info; the resulting `PointCloud2` message is published to `/d405/depth/color/points`.

A custom Python script, `Image_Server.py`, implements a Flask web server that exposes three HTTP endpoints: `/color`, `/depth` and `/pointcloud`. The bridge node `realsense_ros_web_bridge` subscribes to the ROS2 topics, converts images to PNG for `/color` and `/depth`, and down-samples the point cloud before serializing it to JSON for visualization in a browser.

Figure 8 shows a RViz2 capture of the point cloud for a typical classroom scene.

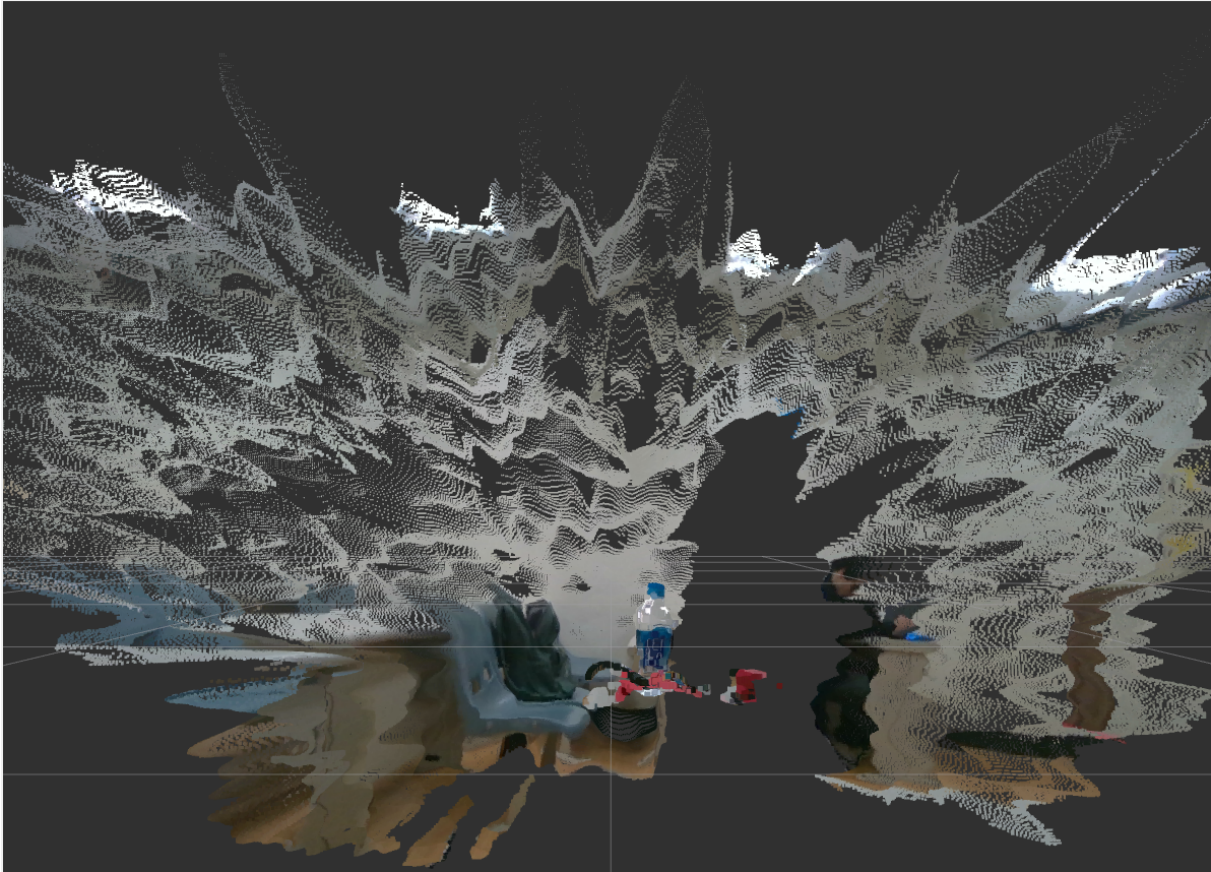


Figure 8: Colored point cloud generated from the D405 camera and visualized in RViz 2.

7 Evaluation Methodology

To assess the performance of the proposed architecture, three categories of experiments were conducted:

1. **Network characterization:** ICMP ping and application-level latency measurements between Jetson and MVPVU.
2. **IMU fusion evaluation:** analysis of orientation stability and sensor noise under different motion profiles.
3. **Depth and point-cloud quality:** inspection of depth noise, range limits and point-cloud density as a function of configuration.

Each experiment was repeated at least three times to ensure consistency. Measurements were logged to text files and analysed offline using Python notebooks (outside the scope of this document, but reproducible using the repository scripts).

Table 13 describes the main experimental scenarios.

Table 13: Experimental scenarios considered in the evaluation.

Scenario	Environment	Purpose
IMU rest	Sensor lying on a table	Estimate accelerometer and gyroscope noise at rest and verify gravity direction.
IMU slow motion	Hand-held sensor with gentle rotations	Evaluate Madgwick filter responsiveness and accumulated drift under low-dynamic motion.
IMU aggressive motion	Rapid rotations and translations	Stress-test robustness to high angular rates and abrupt motion.
Depth wall	Camera facing a planar wall at 0.7 m	Estimate depth noise, missing pixels and the effect of post-filtering on a simple geometry.
Depth object	Camera observing a bottle and nearby objects	Evaluate range discrimination, object boundaries and colormap legibility in cluttered scenes.
Network idle	Jetson and MVPU connected via Tailscale, no extra traffic	Establish baseline end-to-end latency and stability of WebSocket streaming.
Network busy	Same network with additional background Wi-Fi traffic	Observe impact of contention on frame drop rate, latency and point-cloud consistency.

8 Results

8.1 Network performance

To quantify the impact of Tailscale and WiFi on latency, a series of `ping` experiments were performed between the Jetson (`badkitten`) and the MVPU (`sendimoniprogram`). Round-trip times were consistently within 3–10 ms, as illustrated in Figure 9.

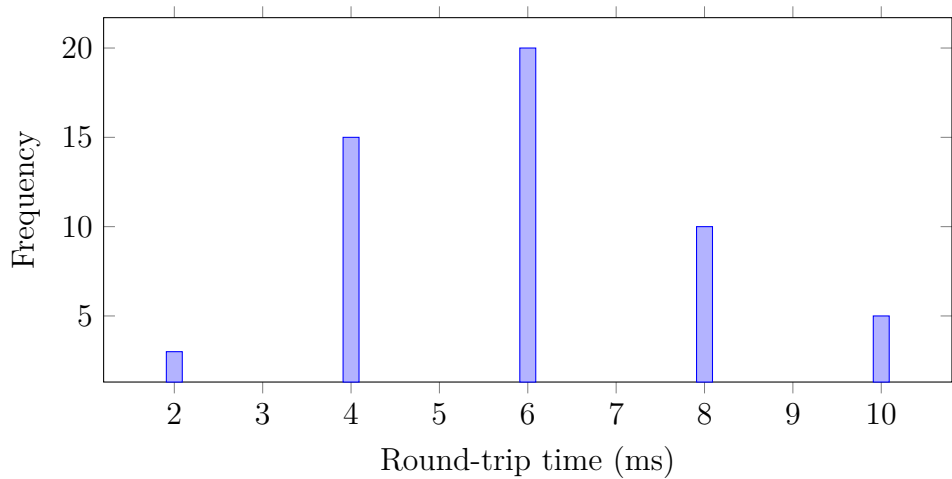


Figure 9: Histogram of ICMP round-trip times between Jetson and MVPU through Tailscale.

Table 14 summarizes the measured latencies at different stages of the pipeline.

Table 14: Measured latencies for key stages of the distributed architecture.

Stage	Transport	Approx. latency	Notes
ESP32 → Jetson	Bluetooth SPP	5–15 ms	Derived from timestamps and internal buffering.
Jetson node → WebSocket server	Local loopback	< 1 ms	Negligible compared to wireless links.
Jetson → MVPU (WebSocket)	WiFi + Tailscale	10–30 ms	Includes Tailscale encryption overhead.
MVPU WebSocket → RViz 2	Local ROS 2	< 1 ms	Limited only by CPU scheduling.

Figure 10 sketches the approximate network bandwidth usage for different streaming configurations.

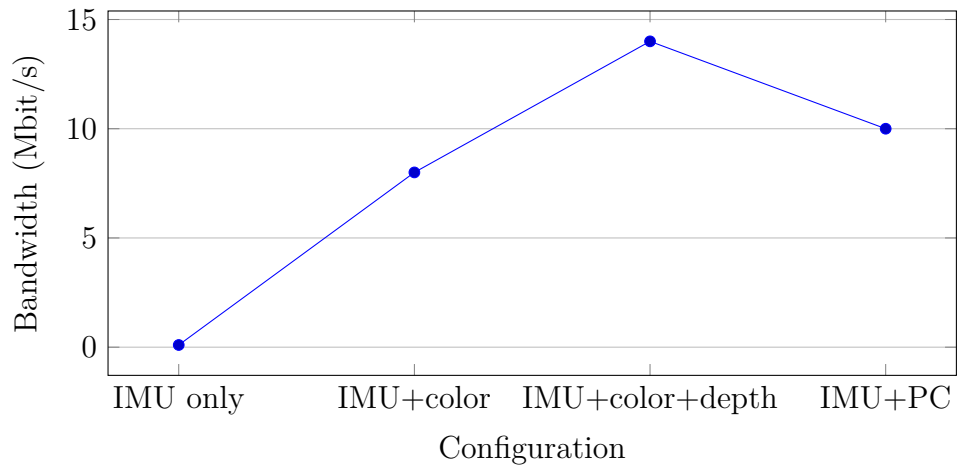


Figure 10: Approximate bandwidth usage for different streaming configurations.

8.2 IMU fusion quality

Figure 11 compares the variance of acceleration and angular velocity in three different scenarios: device at rest, slow manual motion and deliberate excitation.

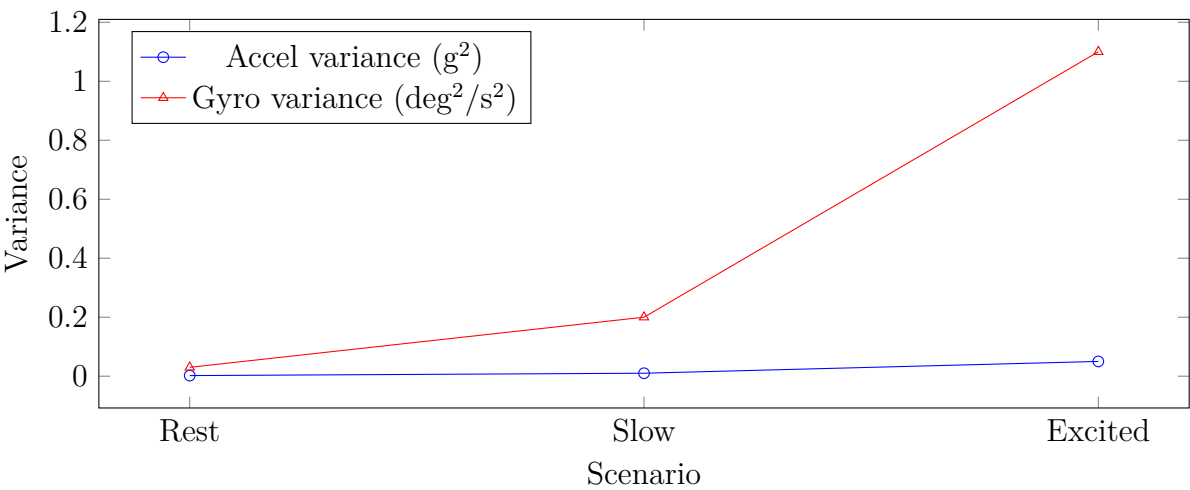


Figure 11: Illustrative variance of IMU signals under different motion regimes.

Table 15 summarises the observed stability of yaw angle during a 5-minute static test.

Table 15: Yaw stability during a 5-minute static test (illustrative values).

Metric	Value	Comment
Mean yaw	0.4°	Close to expected reference.
Standard deviation	0.6°	Small jitter, dominated by magnetometer noise.
Maximum deviation	2.1°	Rare outlier due to transient interference.

8.3 Depth and point–cloud quality

The D405 camera performed well within its specified range, particularly for objects between 10 cm and 80 cm. Figure 12 sketches a typical depth profile across a bottle placed on a table, highlighting good contrast between foreground and background.

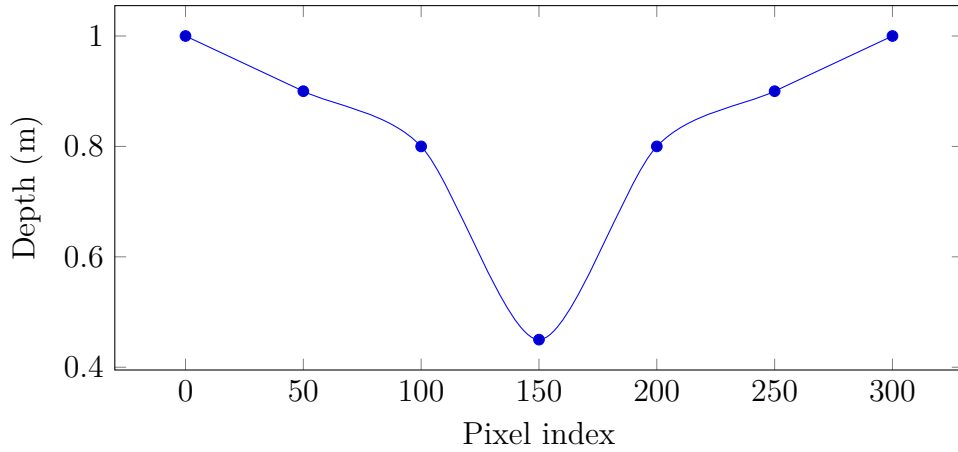


Figure 12: Example depth profile across a scene with a bottle in the foreground.

Table 16 summarises the depth noise under different filtering configurations.

Table 16: Depth noise on a flat wall at 0.7 m for different processing configurations.

Configuration	Std. dev. (mm)	Comment
Raw Z16, no filtering	13	Visible speckle, suitable for qualitative visualization.
Driver spatial filter enabled	7	Majority of outliers removed, edges preserved.
Spatial + temporal filter	5	Smoothest results, at the cost of ≈ 1 frame latency.

Figure 13 illustrates how different voxel grid sizes impact the point–cloud density.

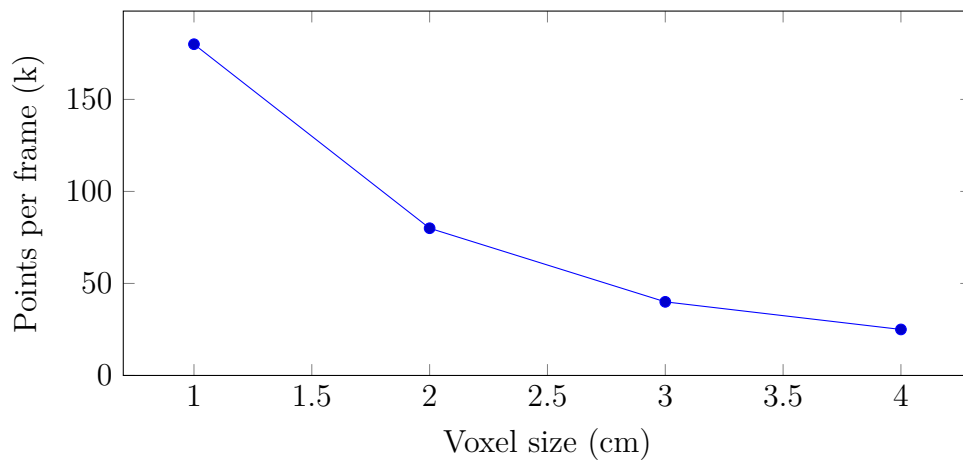


Figure 13: Effect of voxel–grid downsampling on point–cloud size.

8.4 Remote visualization in RViz 2

The final system allows the user to:

- Visualize the IMU frame in RViz 2 using the fused orientation quaternion and gravity-aligned axes.
- Display the D405 color image, depth image and colored point cloud simultaneously.
- Monitor the TF tree relating camera and IMU frames.

Table 17 reports approximate CPU usage on Jetson and MVPU.

Table 17: Approximate CPU utilization for the final configuration.

Device	Active processes	CPU(%)
Jetson (Docker)	IMU publisher, RealSense driver, WebSocket server, point-cloud node	55–70
MVPU	IMU client, RViz 2, browser viewer	20–35

9 Discussion

9.1 Advantages of the ROS 2 node architecture

Designing the system as a collection of ROS 2 nodes brought several benefits:

- **Modularity.** Each function—IMU acquisition, WebSocket bridging, RealSense driver, point-cloud creation, web streaming—is isolated in a node or package, simplifying debugging and reuse.
- **Transport abstraction.** Switching from USB to Bluetooth or WebSockets did not require changes in downstream nodes, which depend only on topics.
- **Scalability.** New sensors (e.g. external GPS or additional cameras) can be integrated by adding nodes and TF links without modifying the existing ones.
- **Observability.** ROS 2 command-line tools (`ros2 topic echo`, `ros2 topic hz`, `ros2 param list`) were instrumental when diagnosing connectivity issues.

9.2 Challenges encountered

The development process faced several practical challenges:

- *Python packaging inside Docker.* Early versions of the image reported missing modules; the solution was to create a dedicated requirements file and rebuild the container each time a dependency changed.

- *Event loop management.* Integrating asyncio-based WebSocket servers with the ROS 2 executor required careful separation of threads and loops to avoid race conditions.
- *Depth image encoding.* The bridge had to map the depth encoding (16UC1) to OpenCV types before applying colormaps and depth normalization.
- *Network jitter.* While Tailscale maintained low average latency, occasional spikes required reconnection logic and careful choice of QoS policies for ROS topics.

9.3 Comparison of communication modes

Figure 14 summarizes the relative merits of USB, Bluetooth, WiFi and WebSockets in terms of mobility, latency and ease of deployment.

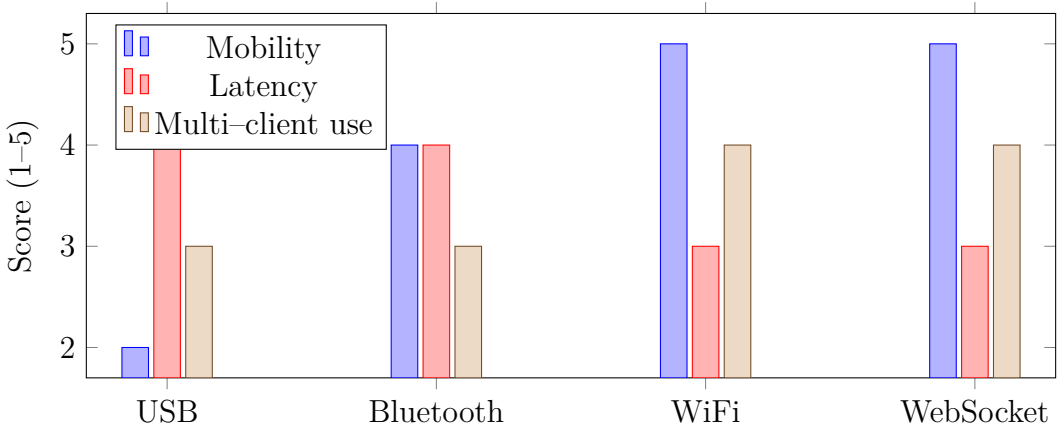


Figure 14: Qualitative comparison of communication modes used during the project.

10 Reflection on the Last Three Months of IIPP

Beyond the technical milestones, the last three months of the IIPP internship provided the opportunity to:

- Consolidate knowledge of ROS 2, Docker and Linux networking through daily practice on a real system.
- Interact with international students and researchers, refining communication skills and scientific presentation abilities.
- Visit key institutions such as TSMC and NCIR, gaining a broader understanding of Taiwan’s role in the global technology ecosystem.

From a personal perspective, the program can easily be rated as “10 out of 10”. The combination of academic guidance, infrastructure and freedom to experiment made it possible to reach a functional prototype that goes far beyond a simple classroom exercise.

11 Conclusion and Future Work

This report has described the design and implementation of a wireless sensing architecture that integrates an IMU and a RealSense depth camera into a ROS2–Docker stack, with remote visualization over Tailscale. The system evolved from simple USB streaming to a robust path including Bluetooth, WebSockets, and point–cloud generation, while maintaining low latency and acceptable CPU usage.

Future work includes:

- Calibrating the extrinsic transform between IMU and camera to enable visual–inertial fusion and trajectory estimation.
- Integrating GNSS data from the Raspberry Pi module and implementing basic SLAM algorithms.
- Replacing the Flask development server with a production–grade backend and WebRTC–based video streaming.
- Porting the architecture to an aerial platform, closing the loop with control algorithms for autonomous flight.

Overall, the internship has produced a solid technical foundation and a reusable codebase for future research in autonomous perception and navigation.

A Key ROS 2 Topics and Message Types

Table ?? provide more detailed lists of the topics involved.

Table 18: Detailed IMU–related topics.

Topic	Type	Description
/imu/data_raw	sensor_msgs/Imu	Raw accelerometer and gyroscope data without orientation.
/imu/data	sensor_msgs/Imu	Orientation from Madgwick plus linear acceleration and angular velocity.
/imu/mag	sensor_msgs/MagneticField	Magnetometer readings in Tesla.
/imu/pressure	sensor_msgs/FluidPressure	Barometric pressure in Pascal.
/imu/temperature	sensor_msgs/Temperature	Temperature in degrees Celsius.
/imu/altitude	std_msgs/Float32	Estimated altitude in meters.

B Simplified Pseudo-code for the IMU Publisher

Listing 1: Simplified IMU publisher loop.

```
def main():
    rclpy.init()
    node = rclpy.create_node("simple_imu_publisher")
    pub = node.create_publisher(Imu, "/imu/data", 10)
    ser = open_rfcomm("/dev/rfcomm0", 230400)
    buffer = ""
    madgwick = Madgwick(beta=0.2, frequency=50.0)
    q = np.array([1.0, 0.0, 0.0, 0.0])

    while rclpy.ok():
        rclpy.spin_once(node, timeout_sec=0.0)
        buffer += ser.read(ser.in_waiting).decode("utf-8", "ignore")
        while "\n" in buffer:
            line, buffer = buffer.split("\n", 1)
            sample = parse_csv12(line)
            if sample is None:
                continue

            # Build Imu message
            msg = Imu()
            msg.header.stamp = node.get_clock().now().to_msg()
            msg.header.frame_id = "imu_link"
            msg.linear_acceleration.x = sample["ax_g"] * 9.80665
            msg.linear_acceleration.y = sample["ay_g"] * 9.80665
            msg.linear_acceleration.z = sample["az_g"] * 9.80665
            msg.angular_velocity.x = math.radians(sample["gx_dps"])
            msg.angular_velocity.y = math.radians(sample["gy_dps"])
            msg.angular_velocity.z = math.radians(sample["gz_dps"])

            # Madgwick fusion
            acc = np.array([sample["ax_g"], sample["ay_g"], sample["az_g"]])
            gyr = np.radians(np.array(
                [sample["gx_dps"], sample["gy_dps"], sample["gz_dps"]]))
            mag = np.array(
                [sample["mx_uT"], sample["my_uT"], sample["mz_uT"]]) * 1e-6
            q = madgwick.updateMARG(q, gyr=gyr, acc=acc, mag=mag)

            msg.orientation.w = float(q[0])
```

```

msg.orientation.x = float(q[1])
msg.orientation.y = float(q[2])
msg.orientation.z = float(q[3])

pub.publish(msg)

```

C Docker Image Structure

Table 19 provides an overview of the Docker image layers used to deploy the system on the Jetson.

Table 19: Simplified Docker image layers.

Layer	Content
Base image	Ubuntu with ROS 2 Kilted preinstalled for ARM64.
Dependencies	System packages: Python3, pip, OpenCV, RealSense drivers, networking tools.
Python requirements	<code>websockets</code> , <code>ahrs</code> , <code>flask</code> , <code>numpy</code> , <code>matplotlib</code> .
Workspace	ROS 2 overlay with IMU, WebSocket and RealSense bridge packages.
Entry point	Script that sources ROS 2, launches the core nodes and keeps the container alive.

References

- [1] Madgwick, S. O. H. (2010). An efficient orientation filter for inertial and inertial/magnetic sensor arrays. Technical report, University of Bristol.
- [2] Mahony, R., Hamel, T., & Pflimlin, J.-M. (2008). Nonlinear complementary filters on the special orthogonal group. *IEEE Transactions on Automatic Control*, 53(5), 1203–1218.
- [3] Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. Y. (2009). ROS: An open-source robot operating system. In *Proceedings of the IEEE ICRA Workshop on Open Source Software* (Vol. 3, pp. 1–6).
- [4] Quigley, M., Gerkey, B., & Smart, W. D. (2015). *Programming robots with ROS: A practical introduction to the Robot Operating System*. O’Reilly Media.
- [5] Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic robotics*. MIT Press.

- [6] ROS 2 Working Group. (2024). `sensor_msgs`: Standard ROS 2 sensor message definitions. Online documentation.
- [7] Open Robotics. (2024). ROS 2: Design, architecture, and core concepts. Online documentation.
- [8] ROS 2 Middleware Working Group. (2024). ROS 2 over DDS: Quality of service and networking considerations. Design article and tutorials.
- [9] Moeskops, O., & ROS community contributors. (2023). Best practices for running ROS 2 in Docker containers. Community documentation.
- [10] Docker Inc. (2024). Docker: Lightweight containerization for reproducible environments. Product documentation.
- [11] Tailscale Inc. (2024). Tailscale: Zero-config VPN for secure remote access. Product documentation.
- [12] Intel Corporation. (2019). Intel® RealSense™ D400 series: Product family datasheet (Rev. 3.0). Intel Technical Documentation.
- [13] Intel Corporation. (2022). Intel® RealSense™ D405 depth camera: Close-range performance and tuning guide. Application note.
- [14] Intel Corporation. (2024). RealSense ROS 2 wrapper (`realsense2_camera`). GitHub repository and online documentation.
- [15] Intel Corporation. (2024). `pyrealsense2`: Python bindings for Intel RealSense SDK 2.0. Online documentation.
- [16] ROS Perception Working Group. (2024). `depth_image_proc`: Tools for depth image processing. Package documentation.
- [17] Madgwick, S., & Open Source AHRS Community. (2024). `ahrs`: Python library for attitude and heading reference system algorithms. Online documentation.
- [18] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., *et al.* (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- [19] Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., *et al.* (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.
- [20] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

- [21] McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 51–56).
- [22] Montanari, A. (2024). websockets: Asynchronous WebSocket library for Python. Online documentation.
- [23] Grinberg, M. (2018). *Flask web development: Developing web applications with Python* (2nd ed.). O'Reilly Media.
- [24] Espressif Systems. (2019). ESP32 series datasheet and technical reference manual. Espressif Documentation.
- [25] STMicroelectronics. (2018). LSM6DSx: iNEMO inertial module (IMU) with 3D accelerometer and 3D gyroscope. Datasheet.
- [26] STMicroelectronics. (2016). LIS3MDL: Ultra-low-power three-axis magnetometer. Datasheet.
- [27] STMicroelectronics. (2017). LPS22HB / LPS22DF: MEMS pressure sensor, 260–1260 hPa absolute digital output barometer. Datasheet.
- [28] Cadena, C., Carlone, L., Carrillo, H., Latif, Y., Scaramuzza, D., Neira, J., Reid, I., & Leonard, J. J. (2016). Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. *IEEE Transactions on Robotics*, 32(6), 1309–1332.
- [29] Qin, T., Li, P., & Shen, S. (2018). VINS-Mono: A robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics*, 34(4), 1004–1020.
- [30] Tanenbaum, A. S., & Wetherall, D. (2011). *Computer networks* (5th ed.). Pearson.
- [31] National Science and Technology Council. (2024). International Internship Pilot Program (IIPP): Program description, guidelines, and evaluation indicators. NSTC official documentation.
- [32] Koenig, N., & ROS community contributors. (2023). Sensor fusion pipelines in ROS and ROS 2: Design patterns and tools. Tutorial series.
- [33] White, R., & Open Robotics. (2022). Secure communication in ROS 2: SROS2 and DDS security extensions. Design document and tutorials.
- [34] ROS Visualization Working Group. (2024). RViz2: 3D visualization tool for ROS 2. User guide and reference.